

1) Quantization

The quantization is the processo of mapping a function Q from $\mathbb{R}$ to a discrete set called *Dictionary*.

$$Q : x \in \mathbb{R} \rightarrow y \in C = \{\hat{x}_1, \hat{x}_2, \dots\} \subset \mathbb{R}$$

where

- C is the Dictionary a subset of $\mathbb{R}$
- $\hat{x}_i$ quantization level (code-word)
- $e = x - Q(x)$: The quantization noise (error)
- $\Theta_i = \{x : Q(x) = \hat{x}_i\}$ the decision region. Defined as non intersecting intervals

Quantization can be seen as an encoding/decoding process:

take signal $x(n)$: the encoding step assigns each sample of $x(n)$ to a quantization level $i(n)$. The decoder associates to each quantization level $i(n)$ the corresponding code word (value).

Performance criteria are composed of **Rate** R and **distortion** D .

By assumption $R = \log_2 L$, which is the

- The rate R is the avg number of bits needed to represent a sample. We define

$$R = \log_2 L$$

- The distortion D is the MSE which in case of a random signal equals to the variance:

$$D = \mathbb{E}[|x(n) - Q(x(n))|^2] = \sigma_Q^2$$

1.1) Uniforma Quantization (UQ)

In uniforma quantizers the input range is divided into $L = 2^b$ equal-sized cells with size $\Delta = A/L$. Each cell is represented by its mid-point i.e. the levels are the centers of the cells.

$$\begin{aligned} \forall i, \Delta^i &= \Delta = A/L \\ t^i &= t^{i-1} + \Delta \\ \hat{x}^i &= \frac{t^i + t^{i-1}}{2} \\ \Theta^i &= \left(\hat{x}^i - \frac{\Delta}{2}, \hat{x}^i + \frac{\Delta}{2} \right) \end{aligned}$$

Unsigned Data

In this case the data is in (a, A) and the ceil function suffices. The thresholds are $(0, \Delta, 2\Delta, \dots, L\Delta)$ and thus $t^i = (i-1)\Delta$.

$$\begin{aligned} \text{Encoder: } i &= \left\lceil \frac{x}{\Delta} \right\rceil \\ \text{Decoder: } \hat{x}^i &= i\Delta - \frac{\Delta}{2} \\ \implies Q(x) &= \Delta \left\lceil \frac{x}{\Delta} \right\rceil - \frac{\Delta}{2} \end{aligned}$$

Unsigned Data

In this case the data is in $(-A/2, A/2)$ and the round function is used. Moreover since 0 must be a quantizer value not a threshold a **odd number of levels is required**

$$\text{Encoder: } i = \text{round}\left(\frac{x}{\Delta}\right)$$

$$\text{Decoder: } \hat{x}^i = i\Delta$$

$$\implies Q(x) = \Delta \text{round}\left(\frac{x}{\Delta}\right)$$

This is called a **mid-tread** quantizer which keeps near 0 values (noise) to 0.

1.2) Rate Distortion (RD) Curve

First recall the uniform distribution:

$$\mathcal{X} \sim u(a, b), \quad f_X(x) = \begin{cases} \frac{1}{b-a} & x \in [a, b] \\ 0 & \text{elsewhere} \end{cases}, \quad \mathbb{E}[\mathcal{X}] = \frac{b+a}{2}, \quad \text{var}(\mathcal{X}) = \frac{(b-a)^2}{12}$$

And the Law of the Unconscious Statistician (LOTUS):

$$\mathbb{E}[g(x)] = \int g(x) f_X(x) dx$$

These allows to calculate the distortion of a uniform RV quantized with UQ:

Let $X \sim U\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$ and $Q(x)$ a UQ with L levels. Then:

$$D = \sigma_Q^2 = \mathbb{E}[(X - \hat{X})^2] = \frac{\Delta^2}{12} = \frac{1}{12} \frac{A^2}{L^2} = \sigma_x^2 2^{-2R}$$

$$SNR = 10 \log_{10} \frac{\mathbb{E}[X^2]}{D} = 10 \log_{10} 2^{2R} \approx 6R$$

Proof:

todo calculation of interval

From here recall that $R = \log_2 L \rightarrow L = 2^R$ and that since X is uniform it has variance $A^2/12$ and thus $\frac{A^2}{12L^2} = \sigma_x^2 / 2^{2R}$.

The SNR is very straight forward: Notice $\mathbb{E}[X^2] = \sigma_x^2$

High Resolution (HR) Quantization

For high resolution we mean $L \rightarrow \infty$

For a region Θ^i the value p_x is constant.

The quantization noise in Θ^i is $\sim u\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right)$ and with the total probability law we have

$$E \sim u\left(-\frac{\Delta}{2}, \frac{\Delta}{2}\right) \rightarrow D = \frac{\Delta^2}{12} = \frac{A^2}{12} 2^{-2R}$$

and the SNR becomes

$$SNR = 10 \log_{10} \frac{\mathbb{E}[X^2]}{D} = 10 \log_{10} \frac{\sigma_X^2}{A^2/12} 2^{2R} \approx 6R - 10 \log_{10} \frac{A^2/4}{3\sigma_X^2} = 6R - 10 \log_{10} \frac{\gamma^2}{3}$$

where γ^2 is the load factor, that is, the ratio between peak power and avg power. With this in mind:

$$D = \frac{\gamma^2}{3} \sigma_x^2 2^{-2R} = K_X \sigma_X^2 2^{-2R}$$

1.3) Optimal Scalar Quantization (SQ)

For a given probability density function $p_X(x)$ we want to find the quantizer (thresholds t^i and levels $\hat{x}^i$) that minimizes the distortion D for a given rate R

Optimal HR Quantizer

Recall the HR hypothesis:

$$L \rightarrow \infty \quad \max_i \Delta_i \rightarrow 0 \quad \forall i, u \in \Theta^i, p_X(x) \approx P_i$$

The RD curve becomes:

$$\sigma_Q^2 = c_X \sigma_X^2 2^{-2R} \quad \text{with } c_X = \frac{1}{12} \left[\int p_U^{1/3}(t) dt \right] \text{ and } U = \frac{X}{\sigma_X}$$

c_X is the shape factor since it only depends on the pdf.
some common shape factors are:

- Uniform: $c_X = 1$
- Gaussian $c_X = \frac{\sqrt{3}}{2} \pi \approx 2.72$

Non HR Quantizer

The solution doesn't have closed analytical formula, but

- There exists necessary conditions
- Lloyd-Max algorithm for local minima

How does the algorithm work?

1. Choose initial dictionary $C^{(0)} = \{\hat{x}_0^i\}_{i=1, \dots, L}$
2. Find best thresholds for given dictionary $\{t^i = \frac{\hat{x}^i + \hat{x}^{i+1}}{2}\}$
3. Find best dictionary for given thresholds $C^{(1)} = \{\hat{x}_{(1)}^i\}$
4. iterate 2. 3. until convergence (stop condition)

The initial dictionary can be chosen:

- Uniform Initialization: $\hat{x}^n = -A_0 + n\Delta \quad \forall n = 1, \dots, L$
- Random initialization: code-words are initialized with the same distribution as data

The (quantizer) decision rule is the nearest neighbor:

$$k = \arg \min_n |x - \hat{x}^n| \rightarrow Q(x) = \hat{x}^k$$

The threshold decision rule is:

$$t^i = \frac{\hat{x}^i + \hat{x}^{i+1}}{2}$$

1.4) Predictive SQ

Quantization alone is not effective for compression of non-sparse data. Often samples have dependencies between each other (image has many adjacent pixels of same color).

This algorithm exploits the sample correlation and reduces the variance by increasing sparsity
But what is a sparse signal?

Definition 1 (Sparse Signal).

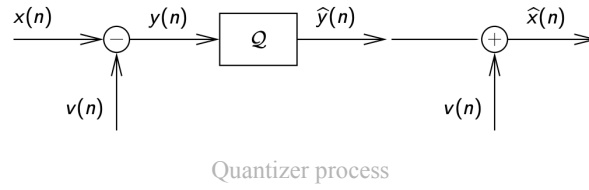
A signal is sparse if most of its components are zero or close to zero

- The variance of a sparse signal is low

- Zero (or close to zero) samples can be neglected (quantized to 0) without introducing distortion
- Natural signals are not sparse but can easily be transformed into one

The basic idea is to define the prediction error as the signal error

$$q(n) = y(n) - \hat{y}(n) = x(n) - v(n) - (\hat{x}(n) - v(n)) = \bar{q}(n)$$



So the SNR can be written as a sum of two terms:

$$SNR = 10\log_{10} \frac{\sigma_X^2}{D} = 10\log_{10} \frac{\sigma_X^2}{\sigma_Y^2} + 10\log_{10} \frac{\sigma_Y^2}{D} = G_P + G_Q$$

Where G_Q is the standard quantizer SNR on the signal y while G_P is the term added which is positive only if the variance of the sparse signal y is lower than the original signal x

Example 2.

Consider a gaussian signal $X(n) \sim \mathcal{N}(0, \sigma^2)$ and $V(n) = X(n-1)$. Moreover suppose that $\mathbb{E}[X(n)X(m)] = \sigma^2 \rho^{|n-m|}$. The sparse signal becomes

$$Y(n) = X(n) - X(n-1)$$

The variance moreover becomes:

$$\begin{aligned} \sigma_Y^2 &= \mathbb{E}[(X(n) - X(n-1))^2] \\ &= \mathbb{E}[X(n)^2 + X(n-1)^2 - 2X(n)X(n-1)] \\ &= 2\sigma^2 - 2\sigma^2\rho \end{aligned}$$

and the prediction gain becomes

$$G_P = 10\log_{10} \frac{\sigma^2}{2(1-\rho)\sigma^2}$$

which is good only if $G_P > 0 \rightarrow \rho > \frac{1}{2}$

Linear Predictor

A linear predictor is a linear combination of the P previous values

$$v(n) = - \sum_{i=1}^P a_i x_{n-i}$$

I like to see it this way $x_{n-1} = x(n-i)$

The prediction error becomes

$$y(n) = x(n) - v(n) = \sum_{i=0}^P a_i x_{n-i}$$

with $a_0 = 1$

The transfer function reduces to

$$A(z) = \sum_{i=0}^P a_i z^{-i} = 1 + a_1 z^{-1} + \dots + a_P z^{-P}$$

Therefore the problem becomes a minimization problem: find values of a_i that minimize the variance:

$$\begin{aligned} \sigma_Y^2 &= \mathbb{E}[Y(n)] = \mathbb{E} \left[X(n) + \sum_{i=1}^P a_i X(n-i) \right]^2 \\ &= \mathbb{E}[X(n)^2] + 2 \sum_{i=1}^P a_i \mathbb{E}[X(n)X(n-i)] + \sum_{i=1}^P \sum_{j=1}^P a_i a_j \mathbb{E}[X(n-i)X(n-j)] \\ &= \sigma_X^2 + 2r^t a + a^t R_X a \end{aligned}$$

where a is the vector with a_i entries and r is the autocorrelation matrix

$$r = [r_X(1) \dots r_X(P)] \quad R_X = \begin{bmatrix} r_X(0) & r_X(1) & r_X(2) & \dots & r_X(P-1) \\ r_X(1) & r_X(0) & r_X(1) & \dots & r_X(P-2) \\ r_X(2) & r_X(1) & r_X(0) & \dots & r_X(P-3) \\ \dots & \dots & \dots & \dots & \dots \\ r_X(P-2) & r_X(P-3) & r_X(P-4) & \dots & r_X(1) \\ r_X(P-1) & r_X(P-2) & r_X(P-3) & \dots & r_X(0) \end{bmatrix}$$

$$r_X(k) = \mathbb{E}\{X(n)X(n-k)\}$$

r definition|

by deriving wrt a we get

$$\frac{\partial \sigma_Y^2}{\partial a} = 2r + 2R_X a \rightarrow a_{opt} = -R_X^{-1} r \rightarrow \sigma_Y^2 = \sigma_X^2 + r^t a_{opt}$$

and the autocorrelation can be estimated as

$$\hat{r}_X(k) = \frac{1}{N} \sum_{n=0}^{N-1-k} X(n)X(n+k)$$

TODO capire meglio

2) Lossless Coding

Lossless coding means to decrease the number of bits needed to encode the data without losing any information: the process is reversible.

The idea is based on variable length coding (VLC) where more probable symbols are encoded in shorter codewords.

Lets start with some notation:

- **Alphabet:** $\mathcal{X} = \{x_1, \dots, x_M\}$ set of symbols to encode
- **Code:** application between $\mathcal{X}$ and $\{0, 1\}$ (set of finite length bit strings)

Digression on fixed length coding (FLC): in FLC all codewords ave the same length, that is M symbols $\rightarrow \lceil \log M \rceil$ bits to encode each symbol.

Example 3 (FLC Text Compression).

An alphabet with 26 symbols is encoded with $\lceil \log 26 \rceil = \lceil 4.7 \rceil = 5$ bits per codeword. This has a compression ratio of 1.

In VLC each codeword c_i has length l_i . This is possible if

- **Decodability Condition:** prefix to distinguish codewords
- **Non-equiprobable symbols**

Example 4 (Example on Decodability Condition).

Consider the following alphabet $\mathcal{X} = \{A, B, C\}$ with the following 4 codes:

Symbol	Code 1	Code 2	Code 3	Code 4
A	0	0	0	0
B	11	10	01	01
C	111	110	011	11

Coding Schemes

- *Code 1 cannot be decoded uniquely: example 111111 can be either BBB or CC*
- *Code 2 is instaneously decodable (prefix condition)*
- *Code 3 is decodable with delay less than or equal to 1 bit (delay counted from last bit)*
- *Code 4 is decodable with an undefined delay*

2.1) Principle of Information Theory

From the 4 codes before we can see that there is a favorite one, based on the following theorem we can choose to focus on **instantaneous codes**

Theorem 5 (McMillan's Theorem).

Decodable codes do not improve performance with respect to instantaneous codes

Theorem 6 (Kraft's Inequality).

There exists a instantaneous code with lengths $\{l_1, \dots, l_M\}$ iff

$$\sum_i 2^{-l_i} \leq 1$$

See [proof](#).

If the equality holds then the code is said to be **optimal**.

Information and Entropy Recap

Suppose symbols $\{x_i\}$ with probabilities p_i .

- The information of a symbol is

$$I(x_i) = -\log_2 p_i = \log_2 \frac{1}{p_i}$$

- The source entropy of $X = \{x_i\}$ is

$$H(X) = -\sum_i p_i \log_2 p_i = -\mathbb{E}[\log_2(p_X(x))]$$

this is the avg uncertainty of X .

- Joint Entropy:

$$H(X, Y) = -\sum_{i,j} p_{i,j} \log p_{i,j}$$

- Conditional Entropy

$$\begin{aligned} H(X|Y) &= \sum_j p_j H(X|Y = y_j) \rightarrow H(X, Y) = H(Y) + H(X|Y) \\ &= H(Y) + H(Y|X) \end{aligned}$$

- Properties:

$$\begin{aligned} H(X) &> 0 \\ H(X, Y) &= H(X) + H(Y|X) = H(Y) + H(X|Y) \\ H(X, Y) &\leq H(X) + H(Y) \text{ (if } \perp) \\ H(X|Y) &\leq H(X) \text{ (if } \perp) \\ H(X) &\leq \log_2 M \text{ (if } X \sim u) \end{aligned}$$

Lagrange's Method

Lagrange's method is used to solve minmax problems with constraints.

Consider a function $f : x \in \mathbb{R}^n \rightarrow \mathbb{R}$

In order to find the maximum or minimum of f subject to the constraint $\phi(x) = 0$, we look for the stationary points of

$$J(x, \lambda) = f(x) + \lambda \phi(x), \quad \lambda \in \mathbb{R}$$

The stationary points are computed by setting to zero all the derivatives of J :

$$\frac{\partial J}{\partial x_i} = 0, \quad \frac{\partial J}{\partial \lambda} = 0$$

Example 7 (Distribution with Maximum Entropy).

The distribution maximizing the entropy of a M -ary discrete r.v. is found applying the Lagrange's method:

$$p^* = \arg \max_p \sum_{i=1}^M p_i \log \frac{1}{p_i}, \quad \sum_i p_i = 1 \rightarrow \phi(x) = \sum_i p_i - 1 = 0$$

Write J :

$$J(p, \lambda) = - \sum_i p_i \log p_i + \lambda \left(\sum_i p_i - 1 \right)$$

Calculate the derivative (specific p_i)

$$\frac{\partial J}{\partial p_i} = - \left(\frac{\log e}{p_i} p_i + \log p_i \right) + \lambda = 0 \rightarrow p_i = \lambda - \log e$$

The max uncertainty is obtained by setting all probabilities equal, that is $p_i = 1/M$.

2.2) Optimal Code

Given a set of M symbols with probabilities p_i we want to find the set of M lengths l_i such that:

- Kraft Inequality is satisfied (prefix code)
- $\forall i = 1, \dots, M, l_i \in \mathbb{N}$
- The avg length $L = \sum_i p_i l_i$ is minimized among all sets of l_i
 $\implies$ constrained minimization problem

$$l^* = \arg \min_l \sum_i p_i l_i \quad \text{subject to} \quad \sum_i 2^{-l_i} = 1$$

TODO Proof

The optimal length, which is also the lower bound, corresponds to the entropy:

$$L^* = - \sum_i p_i \log p_i = H(X)$$

However $H(X) \in \mathbb{R}$ not $\mathbb{N}$.

Taking $l_i = \lceil -\log p_i \rceil$ suffices to find a code such that $H(X) \leq L^* < H(X) + 1$

Proof:

Kraft is satisfied:

$$\begin{aligned} l_i &= -\log p_i + \delta_i \quad \delta_i \in [0, 1) \\ 2^{-l_i} &= p_i 2^{-\delta_i} = \epsilon_i p_i \quad \epsilon_i \in \left(\frac{1}{2}, 1 \right] \\ \sum_i 2^{-l_i} &= \sum_i \epsilon_i p_i \leq \sum_i p_i = 1 \end{aligned}$$

The avg length is:

$$l_i = \lceil -\log p_i \rceil < -\log p_i + 1 \rightarrow p_i l_i = -p_i \log p_i + p_i$$

$$H(X) < \sum_i p_i l_i < \sum_i -p_i \log p_i + p_i$$

$$H(X) \leq L^* < H(X) + 1$$

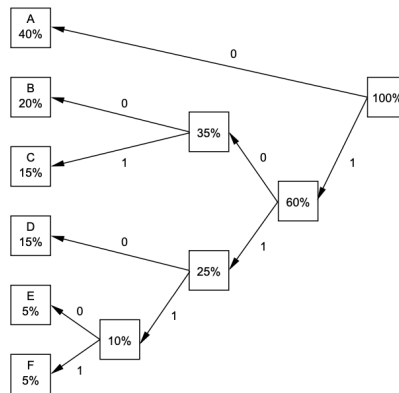
□

Huffman Coding

Huffman discovered how to build the optimal lossless coder for any source with known probabilities.

Algorithm:

1. Create leaf nodes for each symbol, weighted by p_i
2. While there is more than one node:
 - Select two nodes with lowest weights
 - Create a new internal node as their parent
 - Set new node's weight as sum of children's weights
3. Assign '0' to left edges, '1' to right edges
4. Read code for each symbol from root to leaf



Example

This code has avg length $L^* = 2.3$ and entropy $H(X) = 2.246$

As a brief recap consider this:

$$l_i = \lceil \log_2 p_i \rceil \quad H(X) = - \sum_i p_i \log_2 p_i$$

$$H(X) \leq L^* = \sum_i p_i l_i < H(X) + 1$$

Block Coding

Instead of mapping one symbol into one codeword, we can map many subsequent symbols into a single codeword. This gives a better performance as:

Let $X^K = (X_1, X_2, \dots, X_K)$ a block of K symbols. Then

$$H(X^K) = H(X_1, X_2, \dots, X_K) \leq \sum_{i=1}^K H(X_i)$$

Then there are 2 scenarios (L_s is the avg symbol length):

- Symbols not independent:

$$\frac{H(X^K)}{K} < H(X_i) \rightarrow L < H(X^K) + 1 \iff L_s < \frac{H(X^K)}{K} + \frac{1}{K}$$

- Symbols identically distributed (but not necessarily independent):

$$L_s < \frac{H(X^K)}{K} + \frac{1}{K} \leq H(X_i) + \frac{1}{K}$$

These blocks are encoded via Huffman.

Example 8 (BW image with iid pixels).

Let a BW image have probability p of having a black pixel. Then we group pixels into blocks of 2, the probabilities are:

$$P(BB) = p^2 \quad P(BW) = P(WB) = p(1-p) \approx p \quad P(WW) = (1-p)^2 \approx 1-2p$$

from here

$$H(X_1, X_2) = 2H(X) \ll 2$$

Apply the Huffman code to the blocks and by an appropriate code we have

$$L \approx 1 \text{ but } L_s \approx \frac{1}{2}$$

Now define the Entropic Rate:

Definition 9 (Entropic Rate).

When the rate is converging, then

$$\mathcal{H}(X) = \lim_{K \rightarrow \infty} \frac{H(K)}{K} = \lim_{K \rightarrow \infty} L_s^* \leq H(X)$$

Recall the avg length of the optimal code:

$$\begin{aligned} H(X^K) &\leq L^* < H(X^K) + 1 \\ \frac{H(X^K)}{K} &\leq L_s^* < \frac{H(X^K)}{K} + \frac{1}{K} \\ &\downarrow K \rightarrow \infty \\ L_s^* &\rightarrow \mathcal{H}(X) \leq H(X) \end{aligned}$$

where the last steps inequality uses the joint entropy property.

Therefore Huffman code is optimal with larger and larger block sizes. This is practically unobtainable as complexity is exponential with K and probability estimation is hard with bigger blocks.

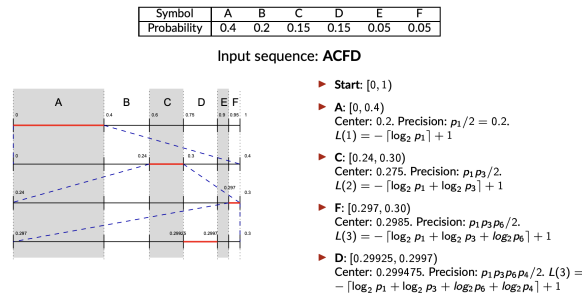
Arithmetic Coding

Arithmetic coding solves the exponential complexity with K increasing. This coding is suboptimal but asymptotically optimal.

$$L \leq H(X^K) + 2 \rightarrow L_s = \frac{L}{K} \xrightarrow{K \rightarrow \infty} \mathcal{H}(X)$$

This is done achieved by **encoding a sequence as the center interval** with arbitrary precision $q \in [0, 1]$ a fractional number. The arithmetic code can encode blocks of any size, even the entirety of the message.

This requires **2 multiplications and 2 sums per symbol**



Example

The avg length can be calculated as (iid symbols):

$$\begin{aligned}
 L(n) &= - \left\lceil \sum_{i=1}^n \log_2 p(x_i) \right\rceil + 1 < - \sum_{i=1}^n \log_2 p(x_i) + 2 \\
 \bar{L}(n) &< \frac{- \sum_{i=1}^n \log_2 p(x_i) + 2}{n} \\
 L &= \mathbb{E}[\bar{L}(n)] < \frac{- \sum_{i=0}^n \mathbb{E}[\log_2 p(x_i)] + 2}{n} \\
 &< \frac{- \sum_{i=0}^n H(X_i) + 2}{n} \\
 &< H(X) + \frac{2}{n} \xrightarrow{n \rightarrow \infty} H(X)
 \end{aligned}$$

For non iid symbols we have

$$L(n) < -\log_2 p(x^n) + 2 \rightarrow L = \frac{\mathbb{E}[L(n)]}{n} < \frac{H(X^n) + 2}{n} \rightarrow \mathcal{H}(X)$$

However $p(x^n)$ is hard to estimate for non iid symbols (if iid $p(x^n) = \prod_i^n p(x_i)$)

Context Based Coding

This approach consists in looking at N_s previous symbols to recognize the context (max $N_c = M^{N_s}$ with M the alphabet) and changes the encoder based on the context. This conditioning reduces the entropy of the source.

2.3) Other Coding Techniques

2.3.1) Exp-Golomb Coding

Universal coding for integer numbers, size (bits) proportional to magnitude

Unsigned Integer

Given int $n \in \mathbb{N}$ the representation consists in

- write $n + 1$ in binary
- use min number of bits: $b = \lfloor \log_2(n + 1) \rfloor + 1$
- place $b - 1$ leading zeroes

Example:

$$\begin{aligned}
 n = 0 &\rightarrow \begin{cases} n + 1 = 1_{10} \rightarrow 1_2 \\ b = \lfloor \log_2 1 \rfloor + 1 = 1 \rightarrow 1 \\ b - 1 = 0 \end{cases} \\
 n = 6 &\rightarrow \begin{cases} n + 1 = 7_{10} = 111_2 \rightarrow 1_2 \\ b = \lfloor \log_2 7 \rfloor + 1 = 3 \rightarrow 00111 \\ b - 1 = 2 \end{cases}
 \end{aligned}$$

Signed Integer

Given int $n \in \mathbb{Z}$ the representation consists in

- Map $\mathbb{Z} \rightarrow \mathbb{N}$ using $m(n) = \begin{cases} 2n - 1 & n > 0 \\ -2n & n \leq 0 \end{cases}$
- Use Exp-Golomb for unsigned integer

Example:

$$n = -3 \rightarrow m(n) = 6 \rightarrow 00111$$

$$n = -6 \rightarrow m(n) = 12 \rightarrow \begin{cases} n + 1 = 13_{10} = 1101_2 \rightarrow 1_2 \\ b = \lfloor \log_2 7 \rfloor + 1 = 4 \\ b - 1 = 3 \end{cases} \rightarrow 0001011$$

2.3.2) Dictionary Based Coding

The idea is to ignore initial statistics and build a dictionary of codewords learned in long term. This allows universal capabilities

Theorem 10 (Asymptotic Optimality Theorem).

For stationary and ergodic sources, the dictionary approach is asymptotically optimal:

With length $n \rightarrow \infty$ the average codeword length L_n converges to the entropy rate $\mathcal{H}$ of the source

Characteristics:

- **Dynamic Dictionary Construction:** Dictionary built on the fly during encoding process
- **Universality:** No prior knowledge of source statistics or probability distributions is required
- **Adaptivity:** adapts to non stationary signals that change over time
- **Practical Efficiency:** used in many formats (zip, gzip, etc)
- **Why:** This approach scales well with long patterns

LZW Encoding (TODO Important non si capisce un cazzo)

It uses a **greedy matching process**:

- Encoder reads symbol by symbol
- For each new input, it checks if the pattern (Prefix W +Next K) already exists
- **Iterative search:** If $W + K$ is found, it continues to read until new pattern is found

Dictionary Evolution and Merging

- **New pattern is found:** Index of longest prefix W is sent to bitstream (output), then $W + K$ is added to dictionary
- **Prefix Merging:** This mechanism merges known prefixes with new suffixes, allowing the dictionary to naturally evolve and "track" local statistics.

EXAMPLE:

Input: 0001000000101000010000010

Alphabet: 0, 1

Initialization: $0 \rightarrow 0$, $1 \rightarrow 1$

- First symbol: 0
The encoder passes through the first symbol: Symbol 0 exists in table (0). Continue search.
Encoder now sees 00: new symbol, adds it to table: $2 \rightarrow 00$. Write in output the codeword of 0:
Output = 0
- Second symbol: 0
We are still at the second 0. 0 is in the table, continue. 00 is in the table (2), continue.
001 is not in table, add it: $3 \rightarrow 001$
Output: 0 2
- Fourth symbol: 1
1 is in table (1), continue. 10 is not in table, add it: $4 \rightarrow 10$
Output: 0 2 1
- Fifth symbol: 0
0 is in table, continue. 00 is in table (2), continue. 000 is new, add it to table: $5 \rightarrow 000$
Output: 0 2 1 2

Here a merge is possible: notice that 2, 3 and 5 have the same prefix 00.

- Seventh symbol. 0
0 is in table, 00 is in table, 000 is in table (2), 0000 is not in table, add it: $5 \rightarrow 0000$
Output: 0 2 1 2 2

3) Transform Coding and JPEG

3.1) Block Coding And Quantization

Recall block coding:

Let X_k be a rv with variance σ_k^2 , the quantizer distortion is: $D_k = c_k \sigma_k^2 2^{-2R_k}$

Now consider a block of rv (not iid): $X = [X_1, \dots, X_M]^T$

We apply a quantization step:

From the **Huang-Schilteiss formula** the optimal rate per block is:

$$R_k^* = \frac{R_{tot}}{M} + \frac{1}{2} \log \left[\frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2} \right]$$

The optimal case diverges from the uniform Resource distribution case (call $\bar{R} = R_{tot}/M$) we get that the single sample distortion equals to the global distortion:

$$D_k^* = c_{GM} \sigma_{GM}^2 2^{-2\bar{R}}$$
$$\mathcal{D}^* = \frac{1}{M} \sum_{k=1}^M D_k = c_{GM} \sigma_{GM}^2 2^{-2\bar{R}}$$

In particular consider the gaussian case (shape factor constant $c_{GM} = c_N = c_X$):

$$\mathcal{D}^* = c_N \sigma_{GM}^2 2^{-2\bar{R}}$$

and if the signals are iid (same variance $\sigma_{GM}^2 \sigma_X^2$ and shape factor) the optimal rate is the uniform allocation. For historical reasons it is called PCM:

$$R_k^* = \bar{R} + \frac{1}{2} \log_2 \underbrace{\left[\frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2} \right]}_{=1} = \bar{R}$$
$$\mathcal{D}_{PCM} = c_X \sigma_X^2 2^{-2\bar{R}}$$

PROOFS

Using uniform resource allocation the global distortion is:

$$\begin{aligned} \mathcal{D} &= \frac{1}{M} \mathbb{E}[\|X - Q(X)\|^2] = \frac{1}{M} \mathbb{E}[(X - Q(X))^T (X - Q(X))] \\ &= \frac{1}{M} \mathbb{E}\left[\sum_{k=1}^M (X_k - Q(X_k))^2\right] = \frac{1}{M} \sum_{k=1}^M \mathbb{E}[(X_k - Q(X_k))^2] \\ &= \frac{1}{M} \sum_{k=1}^M D_k = \frac{1}{M} \sum_{k=1}^M c_k \sigma_k^2 2^{-2R_k} \end{aligned}$$

□

The optimal rate is obtained by using the lagrangian multipliers method:

The constraint is: $\sum_{k=1}^{M-1} R_k \leq R_{tot}$

Then the optimization problem becomes:

$$J(R, \lambda) = \frac{1}{M} \sum_{k=1}^M c_k \sigma_k^2 2^{-2R_k} + \lambda \left(\sum_{k=1}^{M-1} R_k - R_{tot} \right)$$

Find the derivatives:

$$\frac{\partial J}{\partial R_k} = -\frac{2 \ln 2}{M} c_k \sigma_k^2 2^{-2R_k} + \lambda \quad \frac{\partial J}{\partial \lambda} = \sum_{k=1}^{M-1} R_k - R_{tot}$$

Set the gradient to 0 and solve:

$$\begin{aligned} -\frac{2 \ln 2}{M} c_k \sigma_k^2 2^{-2R_k} + \lambda &= 0 \\ -2R_k &= \log_2\left(\frac{M\lambda}{2 \ln 2}\right) + \log_2\left(\frac{1}{c_k \sigma_k^2}\right) \\ R_k^* &= \underbrace{\frac{1}{2} \log_2\left(\frac{2 \ln 2}{M}\right)}_{\lambda'} + \frac{1}{2} \log_2(c_k \sigma_k^2) \end{aligned}$$

Now write it with R_{tot}

$$\begin{aligned} R_{tot} &= \sum_{k=1}^M R_k^* = M\lambda' + \frac{1}{2} \sum_{k=1}^M \log_2(c_k \sigma_k^2) \\ \downarrow \\ \lambda' &= \frac{R_{tot}}{M} - \frac{1}{2M} \sum_{k=1}^M \log_2(c_k \sigma_k^2) = \frac{R_{tot}}{M} - \frac{1}{2} \log_2\left(\prod_{k=1}^M (c_k \sigma_k^2)^{1/M}\right) \\ &= \frac{R_{tot}}{M} - \frac{1}{2} \log_2(c_{GM} \sigma_{GM}^2) \\ \downarrow \\ R_k^* &= \lambda' + \frac{1}{2} \log_2(c_k \sigma_k^2) = \frac{R_{tot}}{M} - \frac{1}{2} \log_2(c_{GM} \sigma_{GM}^2) + \frac{1}{2} \log_2(c_k \sigma_k^2) \\ &= \frac{R_{tot}}{M} + \frac{1}{2} \log_2\left(\frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2}\right) \end{aligned}$$

The solution is the **Huang-Schlteiss formula**:

$$R_k^* = \frac{R_{tot}}{M} + \frac{1}{2} \log \left[\frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2} \right]$$

□

Apply the optimal rate to the single distortion

$$D_k^* = c_k \sigma_k^2 2^{-2R_k^*} = c_k \sigma_k^2 2^{-2\bar{R} - \log_2 \frac{c_k \sigma_k^2}{c_{GM} \sigma_{GM}^2}} = c_{GM} \sigma_{GM}^2 2^{-2\bar{R}}$$

□

Digression on Arithmetic VS Geometric Mean (AM, GM)

The means are defined as:

$$z_{AM} = \frac{1}{M} \sum_{k=1}^M z_k \geq z_{GM} = \sqrt[M]{\prod_{k=1}^M z_k}$$

PROOF

By Jenses Inequality we have:

$$f\left(\frac{1}{M} \sum_{k=1}^M z_k\right) \geq \frac{1}{M} \sum_{k=1}^M f(z_k)$$

with f the log function.

$$\log(z_{AM}) \geq \frac{1}{M} \sum_{k=1}^M \log(z_k) = \log\left(\prod_{k=1}^M z_k^{1/M}\right) = \log(z_{GM}) \rightarrow z_{AM} \geq z_{GM}$$

3.2) Transform Coding

The aim is to make the signal sparse, that is, few large samples (low rate EQ) and many small ones (high EQ). This transforms the id (not iid) signal into blocks with diverse variances (same mean) so to minimize σ_{GM}^2 .

For example a fourier transform of a pure sound is sparse: All zeroes except for f_w .

The operator has the following properties:

- Reversible: $Y = T(X) \iff X = T^{-1}(Y)$
- Input in $\mathbb{R}^M$: an image is $\mathbb{R}^2$
- Input is not sparse
- Output is sparse

We consider $Y = \mathcal{T}X$ where $\mathcal{T}$ is an invertible matrix

- Inverse exists by definition
- This acts as a basis change: basis is set of signals to reconstruct intended signal
- If $\mathcal{T}$ is orthogonal the quantization is the same

So the paradigm becomes:

$$x \rightarrow y = \mathcal{T}x \rightarrow \hat{y} = Q(y) \rightarrow \hat{x} = \mathcal{T}^{-1}\hat{y}$$

Orthogonal Transform

Orthogonal Transform has $\mathcal{T}$ as an orthogonal matrix. This gives many advantages:

- Inverse is immediate: $\mathcal{T}^{-1} = \mathcal{T}^T$
- Is an isometry keeps $\mathcal{L}^2$ norm of any X : $\|\mathcal{T}X\|^2 = \|X\|^2$
- Isometry keeps distortion the same on the output: $D_X = D_Y$

Now consider block coding on id (not iid) gaussian rvs. With orthogonal transform we get:

- Distortion can be applied directly to the quantized output: $D_T = D_Y \stackrel{\text{optimal } R_k}{=} c_{GM,Y} \sigma_{GM,Y}^2 2^{-2R_k}$
- Variance of Y is the AM of X : $\sigma_{AM,Y}^2 = \sigma_X^2$
- AM variance is constant

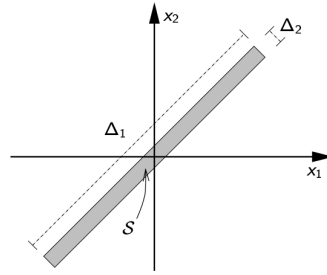
Finally define the **Coding Gain**:

$$G_T = \frac{D_{PCM}}{D_T} = \frac{c_N \sigma_X^2 2^{-2R_k}}{c_N \sigma_{GM,Y}^2 2^{-2R_k}} = \frac{\sigma_{AM,Y}^2}{\sigma_{GM,Y}^2} \geq 1$$

this shows that if the transform reduces the GM variance (with non id samples), then the transform is more efficient.

Example.

Consider a signal where $X_1, X_2 \sim u\left[-\frac{\Delta_1}{2\sqrt{2}}, \frac{\Delta_1}{2\sqrt{2}}\right]$ and have the joint distribution $f_{X_1, X_2}(x_1, x_2) = \begin{cases} \frac{1}{\Delta_1 \Delta_2} & (x_1, x_2) \in S \\ 0 & (x_1, x_2) \notin S \end{cases}$ there S is an oriented rectangle (45°) with dimension $\Delta_1 \times \Delta_2$. With $\Delta_1 \gg \Delta_2$.



Image

Apply uniform quantization (best fit for uniform rv, but not optimal) directly on the samples:

Notice that X_1, X_2 have the same variance $\sigma^2 = \Delta_1^2/24$ and form factor $c = 1$

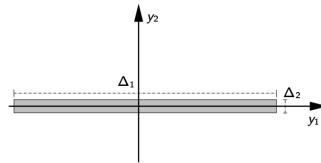
Based on how many bits are assigned to the single rv we get

bits	R	D_1	D_2	D
0	0	σ^2	σ^2	$2\sigma^2$
1	0.5	$\sigma^2/4$	σ^2	$\frac{5}{4}\sigma^2$
1	0.5	σ^2	$\sigma^2/4$	$\frac{5}{4}\sigma^2$
2	1	$\sigma^2/4$	$\sigma^2/4$	$\frac{1}{2}\sigma^2$
2	1	$\sigma^2/16$	σ^2	$\frac{17}{16}\sigma^2$
3	1.5	$\sigma^2/16$	$\sigma^2/4$	$\frac{5}{16}\sigma^2$
4	2	$\sigma^2/16$	$\sigma^2/16$	$\frac{1}{8}\sigma^2$

Table

Apply the following transform $\mathcal{T} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$ (orthogonal since $\mathcal{T}^T \mathcal{T} = I$) then the rvs become:

$Y_i \sim u \left[-\frac{\Delta_i}{2}, \frac{\Delta_i}{2} \right]$ and the joint distribution $f_{Y_1, Y_2}(y_1, y_2) = \begin{cases} \frac{1}{\Delta_1 \Delta_2} & (y_1, y_2) \in S' \\ 0 & (y_1, y_2) \notin S' \end{cases}$ where S' is the rotated surface from before



Image

Now the variances are different $\sigma_1^2 = \Delta_1^2/12 = 2\sigma^2$, $\sigma_2^2 = \Delta_2^2/12 \ll \sigma^2$ and the distortion becomes:

bits	R	D_1	D_2	D
0	0	$2\sigma^2$	$\ll \sigma^2$	$2\sigma^2$
1	0.5	$\sigma^2/2$	$\ll \sigma^2$	$\frac{1}{2}\sigma^2$
2	1	$\sigma^2/8$	$\ll \sigma^2$	$\frac{1}{8}\sigma^2$
3	1.5	$\sigma^2/32$	$\ll \sigma^2$	$\frac{1}{32}\sigma^2$

Table

PROOFS

Inverse is by definition.

□

Isometry:

$$\|\mathcal{T}X\|^2 = (\mathcal{T}X)^T(\mathcal{T}X) = (X^T\mathcal{T}^T)(\mathcal{T}X)X^T(\mathcal{T}^T\mathcal{T})X = X^TX = \|X\|^2$$

And distortion on isometry:

$$D_Y = \frac{1}{M}\mathbb{E}[\|Y - \hat{Y}\|^2] = \frac{1}{M}\mathbb{E}[\|\mathcal{T}(X - \hat{X})\|^2] = \frac{1}{M}\mathbb{E}[\|(X - \hat{X})\|^2] = D_X$$

□

Same distortion:

$$\mathcal{D}_T = \frac{1}{M}\mathbb{E}[\|X - \hat{X}\|^2] = \frac{1}{M}\mathbb{E}[\|\mathcal{T}^{-1}Y - \mathcal{T}^{-1}\hat{Y}\|^2] = \frac{1}{M}\mathbb{E}[\|Y - \hat{Y}\|^2] = D_Y$$

□

Variance equivalences:

$$\begin{aligned}\sigma_X^2 &= \sigma_{AM,X}^2 = \frac{1}{M} \sum_{k=1}^M \mathbb{E}[X_k^2] = \frac{1}{M} \mathbb{E} \left[\sum_{k=1}^M X_k^2 \right] = \frac{1}{M} \mathbb{E}[\|X\|^2] \\ &\quad (\text{recall: } \|Y\|^2 = \|\mathcal{T}X\|^2 = \|X\|^2) \\ &= \frac{1}{M} \mathbb{E}[\|Y\|^2] = \frac{1}{M} \mathbb{E} \left[\sum_{k=1}^M Y_k^2 \right] = \frac{1}{M} \sum_{k=1}^M \mathbb{E}[Y_k^2] \\ &= \sigma_{AM,Y}^2\end{aligned}$$

□

3.3) Practical Implementation of Huang-Schulteiss

HS has a series of limitations:

- Returns negative values
- Returns non-integer values

The **Modified HS algorithm** is very simple:

1. Use HS with all M components
2. If there are negative values, set $R = 0$ of the component and recalculate HS with $M - N$ components
3. When there are only positive values, floor the results
4. Calculate residual rate and allocate eventual residual bits to the results that got rounded the most

Example.

Let there be a gaussian input with 4 components and $R_{tot} = 10$:

$$\sigma_1^2 = 1000, \sigma_2^2 = 100, \sigma_3^2 = 50, \sigma_4^2 = 1$$

The GM is ≈ 47.29

We get:

$$R(1) \approx 4.7, R(2) \approx 3.04, R(3) \approx 2.54, R(4) \approx -0.28 \xrightarrow{\text{negative}} 0$$

Recompute HS with 1,2,3:

$GM \approx 171$

We get (also floor):

$$R(1) \approx 4.61 \rightarrow 4, R(2) \approx 2.95 \rightarrow 2, R(3) \approx 2.45 \rightarrow 2, R(4) = 0 \rightarrow 0$$

Since $4 + 2 + 2 + 0 \neq 10$ we can allocate 2 residual bits to components 1 and 4

$$R(1) = 5, R(2) = 3, R(3) = 2, R(4) = 0$$

The total distortion is:

$$D = \sum D_i = 0.98 + 1.56 + 3.13 + 1 = 6.67$$

The **greedy algorithm** returns the same allocation but is faster:

- Initialization. $R_k = 0 \forall k \in \{0, \dots, M-1\}$ $D_k = \sigma_k^2 \forall k \in \{0, \dots, M-1\}$
 - While $\sum R_k \leq R_{tot}$
 - $l = \arg \max_k D_k$
 - $R_l \leftarrow R_l + 1$
 - $D_l \leftarrow D_l/4$
- This takes R_{tot} iterations

Example.

Consider the same data as before:

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
0	-	0	0	0	0	1000.00	100.00	50.00	1.00
1	1	1	0	0	0	250.00	100.00	50.00	1.00
2	1	2	0	0	0	62.50	100.00	50.00	1.00
3	2	2	1	0	0	62.50	25.00	50.00	1.00
4	1	3	1	0	0	15.63	25.00	50.00	1.00
5	3	3	1	1	0	15.63	25.00	12.50	1.00

Iterations 1-5

Iter	ℓ	R_1	R_2	R_3	R_4	D_1	D_2	D_3	D_4
6	2	3	2	1	0	15.63	6.25	12.50	1.00
7	1	4	2	1	0	3.91	6.25	12.50	1.00
8	3	4	2	2	0	3.91	6.25	3.13	1.00
9	2	4	3	2	0	3.91	1.56	3.13	1.00
10	1	5	3	2	0	0.98	1.56	3.13	1.00

Iterations 6-10

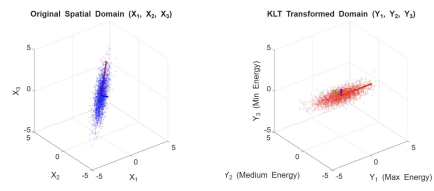
3.4) Orthogonal Transforms For Compression

For gaussian data we use the **KLT** (or principal component analysis (PCA) in ML)

$$\mathcal{T}_{KLT} = E^T$$

Where E is the matrix of eigenvectors of the data covariance matrix.

This is aligning the axis to the direction in which the data has the max variance



Graphical Example

4) Proofs

Kraft Inequality:

Proof of Necessity ($\implies$):

Codewords are already given. Set $L_{\max} = \max_i(l_i)$

Build binary tree with depth $= L_{\max} \rightarrow 2^{L_{\max}}$ number of leaves.

Each codeword with prefix property occupies an entire subtree (since no other codeword must start with same bits). So a codeword with length l_i occupies $2^{L_{\max}-l_i}$ leaves $\rightarrow$ No codeword is a descendant of another, thus these sets of leaves are disjoint.

From here notice that The total number of blocked leaves cannot exceed the available leaves, formally:

$$\sum_{i=1}^N 2^{L_{\max}-l_i} \leq 2^{L_{\max}} \rightarrow \sum_{i=1}^N 2^{-l_i} \leq 1$$

Proof of Sufficiency ($\impliedby$):

We have given $\{l_i\}$ such that $\sum 2^{-l_i} \leq 1$

Again build a binary tree:

The codes can be built as follows:

1. Sort all lengths $l_1 \leq \dots \leq l_N$
2. At each step $k = \{1, \dots, N\}$ pick an available position at depth l_k (decreasing order)
3. Mark descendants of l_k as forbidden. 2^{-l_i} nodes are blocked.

At each step the unavailable nodes are $\sum_{i=1}^{k-1} 2^{-l_i}$. Therefore we have $1 - \sum_{i=1}^{k-1} 2^{-l_i}$ available nodes and by Kraft's

Inequality we have

$$\sum_{i=1}^N 2^{-l_i} = \sum_{i=1}^{k-1} 2^{-l_i} + 2^{-l_k} \leq 1 \rightarrow 1 - \sum_{i=1}^{k-1} 2^{-l_i} \geq 2^{-l_k}$$

Since the remaining free capacity is at least 2^{-l_k} , there must exist at least one free node at depth l_k where the k-th codeword can be placed.

□